

LangGraph

1) What is LangGraph? What problem does it aim to solve compared to a simple linear chain of LLM calls?

LangGraph is a **graph-based orchestration framework for LLM workflows**.

It solves the limitation of linear chains by supporting branching, looping, conditional logic, and shared state, enabling complex and **dynamic AI pipelines**.

2) What are the core concepts of a LangGraph workflow (e.g., nodes, edges, state)?

Nodes → units of work (LLM calls, tools, or functions)

Edges → define data flow between nodes

State / Context → **shared memory** that passes information across nodes

3) Describe how a “node” functions in a LangGraph graph.

A node performs a **single operation** — such as an LLM call, API request, or function execution — and sends its output to the next node.

4) Describe how an “edge” functions in a LangGraph graph.

An edge controls how outputs from one node become inputs for another, defining the data and execution flow of the graph.

5) How is “state” or “context” managed in LangGraph?

LangGraph maintains a shared state object that **all nodes can read and update, enabling persistent data** (like memory or history) throughout the workflow.

6) Give a simple example: build a LangGraph that takes user input, does an LLM call, then a function call. What nodes/edges would you define?

Nodes:

- Input Node → gets user text
- LLM Node → generates summary
- Function Node → formats the result

Edges: Input → LLM → Function

7) What are some typical use-cases where you'd prefer LangGraph over simpler workflows?

- RAG (Retrieval-Augmented Generation) pipelines
- Chatbots with memory
- **Multi-step reasoning agents**
- Automated AI workflows
- Document summarization or data processing

8) How do you handle branching logic in LangGraph (e.g., different edges based on output of a node)?

Use conditional edges — the output of a **node decides which next node runs** (e.g., success → summary node, fail → retry node).

9) What is the difference between synchronous vs asynchronous node execution in LangGraph?

Synchronous: runs step-by-step, **one node at a time.**

Asynchronous: runs independent **nodes in parallel, improving speed and efficiency.**

10) How do you define reusable graph components or sub-graphs in LangGraph?

Group related nodes into a **subgraph or create reusable graph templates/functions** to use across multiple workflows.

11) How do you import or integrate custom functions/tools (e.g., a data API) into LangGraph nodes?

Wrap **external functions or APIs as custom nodes** with clear inputs/outputs, then connect them through edges.

12) How would you monitor the execution of a LangGraph graph? What logging or visualization tools might you use?

Use `.visualize()` or `.to_dot()` for graph visualization.

Enable logging, metrics, or tracing tools to monitor execution flow, node errors, and performance.

13) What are the benefits of using a graph-based orchestration framework like LangGraph in production?

- Modular and scalable design
- Easy debugging and clear visualization
- **Supports branching, loops, and dynamic control flow**
- Reusable and maintainable components
- Better state management and observability

14) What are the limitations or risks of using LangGraph (compared to simpler workflows)?

- Higher setup complexity and learning curve
- More complex debugging for large graphs
- Requires careful state and concurrency management
- Overhead for **simple one-step tasks**

15) How would you explain LangGraph to a non-technical stakeholder (e.g., product manager) in terms of what it brings?

LangGraph is like a flowchart for AI workflows — **it connects LLM steps, tools, and data sources logically**, helping teams automate complex reasoning tasks in a visual and maintainable way.

16) How do you handle error-handling or retry logic in LangGraph nodes?

Use error-handling nodes or **define retry policies for nodes**. You can catch exceptions, retry failed nodes, or redirect errors to fallback paths.

17) How do you implement caching of outputs from nodes to avoid repeated expensive LLM/tool calls?

Use node-level caching — **store previous inputs and outputs in a cache** (e.g., Redis, local DB). If the same input repeats, return cached results instead of re-calling the model.

18) How would you design a LangGraph workflow that handles batching of inputs (e.g., multiple users or items processed in parallel)?

Use batch-processing nodes and parallel edges.

Each node processes a batch (list of items), and **independent batches can run asynchronously for efficiency.**

19) How do you integrate a vector-store retrieval (for RAG) into LangGraph—what nodes and edges would you define?

Nodes:

- Input Node – get query
- Retrieval Node – fetch docs from vector store
- LLM Node – combine query + docs → answer

Edges: Input → Retrieval → LLM

20) How would you design a LangGraph graph to support multi-turn conversation with memory across turns?

Use a persistent state to store chat history.

Each turn updates the state, **which the LLM node reads to keep context across messages.**

21) How do you guard against prompt-injection or malicious input in a workflow built with LangGraph?

Add input validation, sanitization, and filtering nodes.

Limit model instructions and **use guardrails or moderation APIs to detect unsafe inputs.**

22) How do you manage secrets/API keys within a LangGraph workflow (for tool calls)?

Store keys in environment variables or secret managers (e.g., AWS Secrets Manager).

Never hardcode secrets; pass them via **secure config objects** to nodes.

23) How would you version control or deploy LangGraph graphs in a production environment?

Use **Git for version control**.

Store graphs as JSON/YAML configs or Python modules, then deploy via CI/CD pipelines or Docker containers.

24) How would you measure performance (latency, cost, throughput) of a LangGraph workflow and optimise it?

Add logging and **metrics per node**.

Measure latency, token usage, and cost, then optimize by caching, parallelization, or simplifying prompts.

25) Explain how you would debug a failing or stuck node within a LangGraph graph.

Enable debug logging to trace inputs/outputs.

Use **visualization tools to locate the stuck node** and inspect its state and error stack.

26) How do you handle state persistence across graph executions—for example, if you want to resume from a previous state?

Persist state objects in a database or file store.

When restarting, load the previous state and **continue execution from the last completed node**.

27) How would you implement conditional branching in the graph based on output of an upstream node (e.g., if retrieval found docs vs not found)?

Use if-else edges — check the output condition and route flow to either a “Found” node or a “Fallback” node.

28) How do you implement loops or iterative workflows in LangGraph (for example: keep calling model until a quality threshold is met)?

Define loop edges that re-route to the same node if a condition isn't met.

Track iterations in the state to **avoid infinite loops**.

29) How do you integrate external systems (databases, APIs, micro-services) as nodes/tools within LangGraph?

Create custom function nodes that wrap external APIs or DB queries, with input/output mappings to **connect with other graph nodes**.

30) Explain how you'd handle concurrency/multi-tenant workflows using LangGraph (many users, many graphs running at same time)?

Use asynchronous execution and isolated states per user/session. Deploy with **task queues or worker threads** (e.g., Celery, Ray) for scalable multi-user processing.

Scenario Based Questions

31) You need to design a workflow where a user question triggers an LLM call and then a function call. How would you structure this in LangGraph?

A: I'd create an LLM node that processes the user query, then connect it to a function node so the **model output directly triggers the function call.**

32) You want your workflow to branch depending on the LLM output (e.g., positive vs negative sentiment). How would you implement this in LangGraph?

A: I'd use conditional edges that evaluate the LLM's output and route execution to **different nodes based on sentiment type.**

33) You have a long document and you want to chunk it, embed it, then answer questions using those chunks. How would you build this in LangGraph?

A: I'd create nodes for document loading, chunking, embedding, and retrieval, **connecting them to a final LLM answer node** for RAG-style querying.

34) You want to reuse a sub-graph component (e.g., document ingestion) in multiple workflows. How would you define and integrate that in LangGraph?

A: I'd modularize the ingestion logic as a **subgraph and import it into other workflows** for consistency and reusability.

35) You need to maintain context/state across multiple nodes in LangGraph for a multi-turn conversation. How would you handle that?

A: I'd use a shared memory or state object that gets updated and passed between **nodes to preserve context.**

36) You're integrating a custom tool (e.g., database query) as a node in LangGraph. How would you set that up?

A: I'd wrap the tool logic as a Python function, register it as a node, and define its **inputs and outputs for smooth integration**.

37) You need to monitor execution of your LangGraph graph and log each node's output/state. How would you do it?

A: I'd use LangGraph's built-in callback system or a **custom logger to capture node execution** details and outcomes.

38) You want to execute two nodes in parallel (e.g., call two APIs simultaneously) in LangGraph. How would you design this?

A: I'd define both nodes as **parallel branches and merge their outputs** later in a join node.

39) You need to implement caching of outputs in a LangGraph node to avoid repeated expensive calls. How would you handle that?

A: I'd use an **input-based hash key for caching node** outputs and retrieve results from cache when available.

40) You want to handle errors or retries in LangGraph (e.g., a tool call fails). How would you implement this?

A: I'd include **retry logic in nodes** and add fallback paths that activate when errors occur.

41) A workflow must route dynamically: if retrieval found docs, go one path; else go another. How would you model this branching in LangGraph?

A: I'd use a conditional router node that **checks retrieval results** and directs flow accordingly.

42) You need to persist graph execution state so you can resume later. How would you implement state persistence in LangGraph?

A: I'd serialize the execution state to a database or file, then reload it when **resuming the workflow**.

43) You have multiple users executing the same graph concurrently. How do you isolate user-specific state in LangGraph?

A: I'd use **session-based state isolation** so each user's context and memory are kept separate.

44) You want to version control your LangGraph graph definitions and deploy to production. What strategy would you follow?

A: I'd store graph definitions in **Git, implement CI/CD for testing, and deploy stable** tagged versions.

45) You want to evaluate latency and cost of each node in your LangGraph workflow. How would you measure and optimise?

A: I'd log timestamps and token usage for each node, then identify and optimize high-cost areas.

46) You need to protect against prompt-injection or malicious input in a LangGraph workflow. How would you defend?

A: I'd sanitize user inputs and use strict system prompts or guardrails to **block harmful instructions**.

47) You're building a retrieval-augmented generation (RAG) workflow in LangGraph. What nodes and edges would you define?

A: I'd include document loader, chunker, embedder, retriever, and LLM answer nodes **connected sequentially**.

48) You need to embed a vector store lookup tool into your LangGraph graph. How would you integrate it?

A: I'd define a vector store node that takes embeddings as input and returns relevant document IDs or chunks.

49) You want multi-turn conversation with memory in LangGraph: user asks, bot remembers, user asks again. How would you set this up?

A: I'd **attach a memory module** to the conversation flow so previous context is stored and recalled.

50) You want to build loops in LangGraph (e.g., retry until score above threshold). How would you design that graph?

A: I'd define a loop edge that **re-executes nodes** until a stopping condition is satisfied.

51) Your graph must call external microservices (e.g., CRM API) as tools. How would you add and manage these in LangGraph?

A: I'd define **API call nodes as tools** and configure authentication and retry mechanisms for reliability.

52) You need conditional branching based on user input (e.g., select tool A vs tool B). How would you implement this in LangGraph?

A: I'd use an intent-detection node followed by conditional edges routing to **different tool nodes**.

53) You want to design sub-graphs which can be reused in other graphs (e.g., document loader + chunk + embed). How would you build and import them?

A: I'd define modular subgraphs with clear inputs/outputs and import them wherever needed.

54) You must secure secrets/API keys within your LangGraph workflow nodes. How would you handle secret management?

A: I'd use environment variables or a **secret manager to inject credentials securely**.

55) You need to monitor and visualise the node graph structure at runtime for debugging. How would you enable that?

A: I'd use LangGraph's visualization **API or integrate a dashboard** that shows real-time graph state.

56) You want to cache outputs of a LangGraph node across users to save cost. How would you configure this caching?

A: I'd use a shared cache like Redis and key results **based on normalized inputs**.

57) You want your LangGraph graph to scale horizontally for many users. How would you design state isolation and concurrency?

A: I'd make nodes **stateless and manage user-specific states** externally with unique session IDs.

58) You need to build multi-modal input handling (text + image) in LangGraph. What nodes/edges would you define?

A: I'd add a text-processing node and an image-analysis node, merging their **results in a reasoning node**.

59) You want to create an agent-style workflow in LangGraph (tool selection + execution + LLM reasoning). How would you define it?

A: I'd define an **LLM controller node** that decides which tool node to call and loops until the task completes.

60) You need to test your LangGraph workflows with unit tests, mocks, and profiling. How would you set up your testing strategy?

A: I'd write **unit tests for each node**, mock tool outputs, and use profiling tools to measure performance.

Langgraph Interview Questions

- 1) What is LangGraph? What problem does it aim to solve compared to a simple linear chain of LLM calls?
- 2) What are the core concepts of a LangGraph workflow (e.g., nodes, edges, state)?
- 3) Describe how a “node” functions in a LangGraph graph.
- 4) Describe how an “edge” functions in a LangGraph graph.
- 5) How is “state” or “context” managed in LangGraph?
- 6) Give a simple example: build a LangGraph that takes user input, does an LLM call, then a function call. What nodes/edges would you define?
- 7) What are some typical use-cases where you’d prefer LangGraph over simpler workflows?
- 8) How do you handle branching logic in LangGraph (e.g., different edges based on output of a node)?
- 9) What is the difference between synchronous vs asynchronous node execution in LangGraph?
- 10) How do you define reusable graph components or sub-graphs in LangGraph?
- 11) How do you import or integrate custom functions/tools (e.g., a data API) into LangGraph nodes?
- 12) How would you monitor the execution of a LangGraph graph? What logging or visualization tools might you use?
- 13) What are the benefits of using a graph-based orchestration framework like LangGraph in production?
- 14) What are the limitations or risks of using LangGraph (compared to simpler workflows)?
- 15) How would you explain LangGraph to a non-technical stakeholder (e.g., product manager) in terms of what it brings?
- 16) How do you handle error-handling or retry logic in LangGraph nodes?

- 17) How do you implement caching of outputs from nodes to avoid repeated expensive LLM/tool calls?
- 18) How would you design a LangGraph workflow that handles batching of inputs (e.g., multiple users or items processed in parallel)?
- 19) How do you integrate a vector-store retrieval (for RAG) into LangGraph—what nodes and edges would you define?
- 20) How would you design a LangGraph graph to support multi-turn conversation with memory across turns?
- 21) How do you guard against prompt-injection or malicious input in a workflow built with LangGraph?
- 22) How do you manage secrets/API keys within a LangGraph workflow (for tool calls)?
- 23) How would you version control or deploy LangGraph graphs in a production environment?
- 24) How would you measure performance (latency, cost, throughput) of a LangGraph workflow and optimise it?
- 25) Explain how you would debug a failing or stuck node within a LangGraph graph.
- 26) How do you handle state persistence across graph executions—for example, if you want to resume from a previous state?
- 27) How would you implement conditional branching in the graph based on output of an upstream node (e.g., if retrieval found docs vs not found)?
- 28) How do you implement loops or iterative workflows in LangGraph (for example: keep calling model until a quality threshold is met)?
- 29) How do you integrate external systems (databases, APIs, micro-services) as nodes/tools within LangGraph?
- 30) Explain how you'd handle concurrency/multi-tenant workflows using LangGraph (many users, many graphs running at same time).

- 31) You need to design a workflow where a user question triggers an LLM call and then a function call. How would you structure this in LangGraph?
- 32) You want your workflow to branch depending on the LLM output (e.g., positive vs negative sentiment). How would you implement this in LangGraph?
- 33) You have a long document and you want to chunk it, embed it, then answer questions using those chunks. How would you build this in LangGraph?
- 34) You want to reuse a sub-graph component (e.g., document ingestion) in multiple workflows. How would you define and integrate that in LangGraph?
- 35) You need to maintain context/state across multiple nodes in LangGraph for a multi-turn conversation. How would you handle that?
- 36) You're integrating a custom tool (e.g., database query) as a node in LangGraph. How would you set that up?
- 37) You need to monitor execution of your LangGraph graph and log each node's output/state. How would you do it?
- 38) You want to execute two nodes in parallel (e.g., call two APIs simultaneously) in LangGraph. How would you design this?
- 39) You need to implement caching of outputs in a LangGraph node to avoid repeated expensive calls. How would you handle that?
- 40) You want to handle errors or retries in LangGraph (e.g., a tool call fails). How would you implement this?
- 41) A workflow must route dynamically: if retrieval found docs, go one path; else go another. How would you model this branching in LangGraph?
- 42) You need to persist graph execution state so you can resume later. How would you implement state persistence in LangGraph?
- 43) You have multiple users executing the same graph concurrently. How do you isolate user-specific state in LangGraph?
- 44) You want to version control your LangGraph graph definitions and deploy to production. What strategy would you follow?
- 45) You want to evaluate latency and cost of each node in your LangGraph workflow. How would you measure and optimise?

- 46) You need to protect against prompt-injection or malicious input in a LangGraph workflow. How would you defend?
- 47) You're building a retrieval-augmented generation (RAG) workflow in LangGraph. What nodes and edges would you define?
- 48) You need to embed a vector store lookup tool into your LangGraph graph. How would you integrate it?
- 49) You want multi-turn conversation with memory in LangGraph: user asks, bot remembers, user asks again. How would you set this up?
- 50) You want to build loops in LangGraph (e.g., retry until score above threshold). How would you design that graph?
- 51) Your graph must call external microservices (e.g., CRM API) as tools. How would you add and manage these in LangGraph?
- 52) You need conditional branching based on user input (e.g., select tool A vs tool B). How would you implement this in LangGraph?
- 53) You want to design sub-graphs which can be reused in other graphs (e.g., document loader + chunk + embed). How would you build and import them?
- 54) You must secure secrets/API keys within your LangGraph workflow nodes. How would you handle secret management?
- 55) You need to monitor and visualise the node graph structure at runtime for debugging. How would you enable that?
- 56) You want to cache outputs of a LangGraph node across users to save cost. How would you configure this caching?
- 57) You want your LangGraph graph to scale horizontally for many users. How would you design state isolation and concurrency?
- 58) You need to build multi-modal input handling (text + image) in LangGraph. What nodes/edges would you define?
- 59) You want to create an agent-style workflow in LangGraph (tool selection + execution + LLM reasoning). How would you define it?
- 60) You need to test your LangGraph workflows with unit tests, mocks, and profiling. How would you set up your testing strategy?